

< DEBUGGING_CONSCIENTE />

Aprendendo com o ERRO

// Domine a Anatomia do Bug

O GRITO DA MÁQUINA

Transformando pânico em diagnóstico técnico

ESTAGIÁRIO

Pânico Visual

Fecha o console, entra em pânico e altera o código ao acaso (vibe-coding).

ENGENHEIRO

Análise Fria

Lê o Stack Trace como um mapa. Cada erro é uma coordenada para a solução.

A CAUSA RAIZ

O fim da tentativa e erro

- // Não mude o código sem entender o porquê.
- // Copie o Stack Trace (o erro vermelho longo).
- // Use o Gemini com o prompt: **"Explica-me a causa raiz deste erro."**
- // IA correlaciona o erro com seu código.

```
Uncaught TypeError: Cannot read properties of  
undefined (reading 'map')
```

```
at Dashboard.js:45:12
```

```
at App.js:12:3
```

```
// A IA lê este caos e traduz para você.
```

O CANVAS DE DEBUG



PROMPT ESTRUTURADO:

"Onde é que a lógica falhou para gerar um Null Pointer Exception? Analise este contexto."

// Explicação detalhada da falha.

// Correção sugerida com fundamentação.

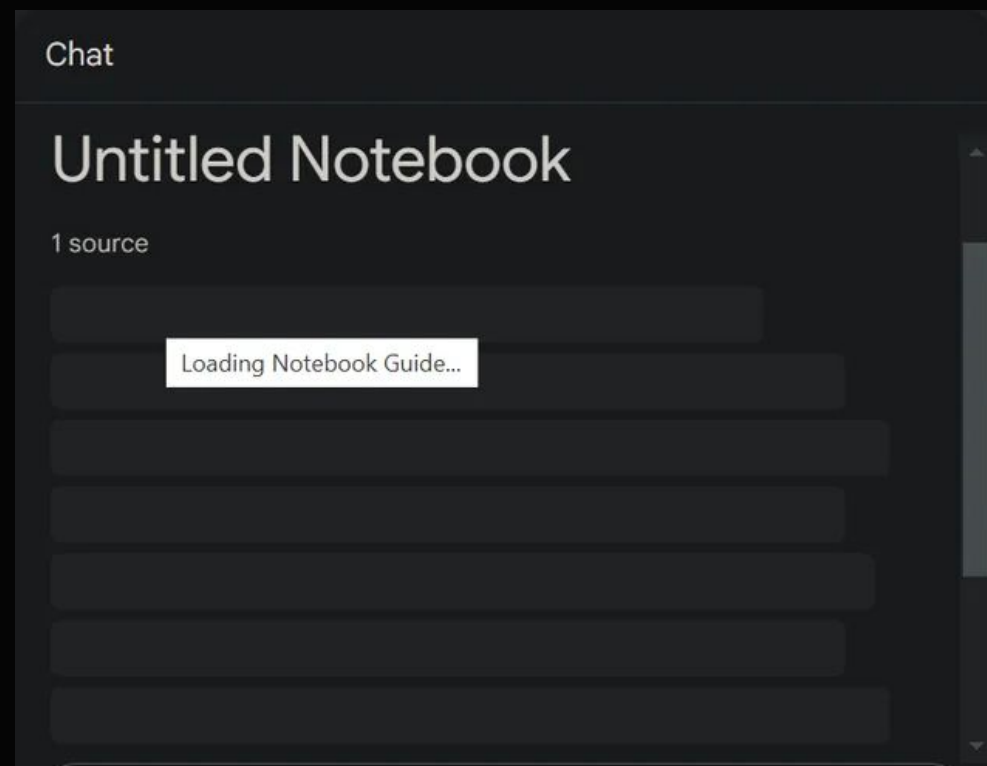
// Prevenção para erros futuros.

SEU GUIA DE ERROS

NotebookLM

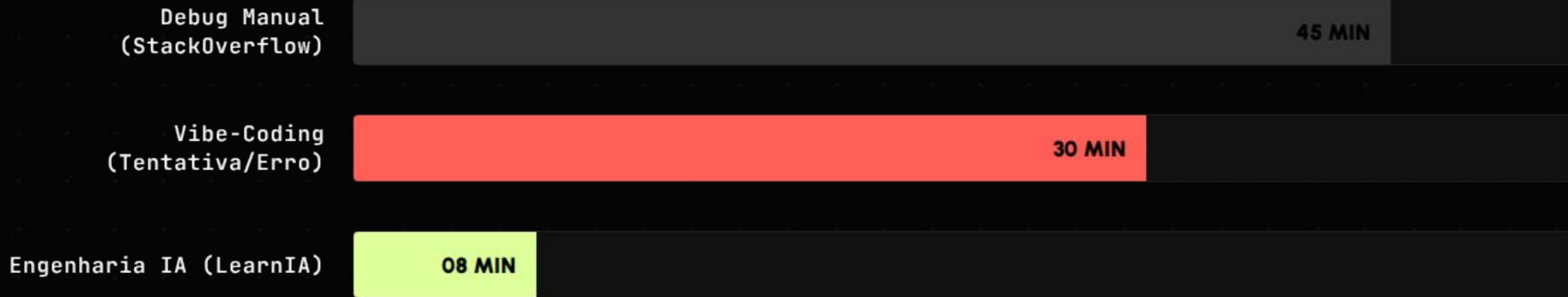
Carregue PDFs de "Erros Comuns em React/Node". Quando um bug bizarro surgir, pergunte ao seu guia privado.

// Correlaciona console + docs oficiais.



TEMPO DE RESOLUÇÃO

*O impacto **real** da análise preditiva*



Redução de 82% no tempo de debug através da análise de causa raiz.

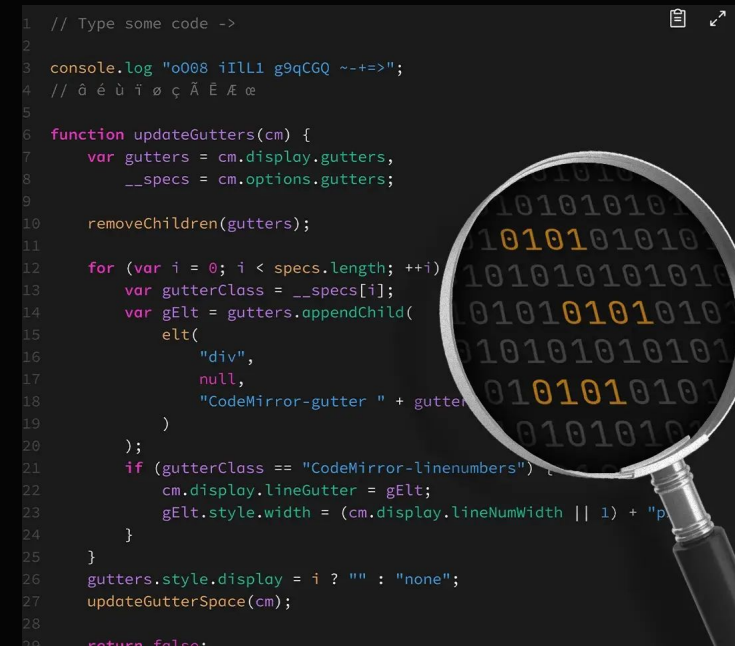
O MÉTODO DO DETETIVE

Debugging não é sobre consertar código. É sobre entender “por que quebrou?”

// Paciência Analítica.

// Leitura de Contexto.

// Validação Socrática.

A screenshot of a code editor with a dark theme. The code is in JavaScript and includes comments in Portuguese. A magnifying glass is positioned over the code, focusing on a specific line. The code is as follows:

```
1 // Type some code ->
2
3 console.log "o008 iIlLl g9qCGQ ~-+=>";
4 // ã é ù î ø ç Å Æ æ
5
6 function updateGutters(cm) {
7   var gutters = cm.display.gutters,
8     __specs = cm.options.gutters;
9
10  removeChildren(gutters);
11
12  for (var i = 0; i < specs.length; ++i)
13    var gutterClass = __specs[i];
14    var gElt = gutters.appendChild(
15      elt(
16        "div",
17        null,
18        "CodeMirror-gutter " + gutterClass
19      )
20    );
21    if (gutterClass == "CodeMirror-linenumbers") {
22      cm.display.lineGutter = gElt;
23      gElt.style.width = (cm.display.lineNumWidth || 1) + "px";
24    }
25  }
26  gutters.style.display = i ? "" : "none";
27  updateGutterSpace(cm);
28
29  return false;
```

The magnifying glass is focused on the line: `gElt.style.width = (cm.display.lineNumWidth || 1) + "px";`. The background of the code editor is dark, and the text is in a light color. The magnifying glass has a silver frame and a black handle.

MÉTODO DO PATINHO

**“Vou explicar te a minha lógica,
interrompe-me se ver algo estranho”**

// Rubber Ducking com IA

A IA atua como seu par, validando se o que está na sua cabeça é o que está no código.

DESAFIO: AUTÓPSIA



IDENTIFICAR

Encontre os 3 bugs propositais no console.



EXPLICAR

Use Gemini para explicar o motivo técnico da falha.



CORRIGIR

Aplique a solução e valide o que aprendeu.



20 MINUTOS DE FOCO TOTAL

MINDSET DO DETETIVE

Antes de corrigir, responda:

- // Eu li o Stack Trace completo?
- // Eu entendi qual linha exatamente falhou?
- // Eu sei o motivo técnico por trás desse erro?
- // Como eu poderia ter evitado este erro com Typescript?





PERGUNTAS?

Obrigado por sua atenção, Engenheiro.

PRÓXIMA AULA: ENGENHARIA REVERSA